Real-world (a real approval pause) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `approval_pause_practice.py` : this is the exact `interrupt()` pattern the Build Task needs at its final gate: the graph calls `interrupt()` with the draft and its sources, execution genuinely stops there (nothing after it runs), and only `Command(resume=decision)` continues it — with a human's decision now part of the state. **If not:** the Build Task's approval step would be the first `interrupt()` you ever wired to a real search-and-draft graph.

Uses the same `practice/` setup as the Basic exercise (Doc08's files, `docs/` and `kb.py`).

Solution

```
# approval_pause_practice.py
from typing import TypedDict

from langchain_openai import ChatOpenAI
from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import END, START, StateGraph
from langgraph.types import Command, interrupt

from kb import collection
from retriever import retrieve

model = ChatOpenAI(model="gpt-4o-mini", temperature=0)

class ApprovalState(TypedDict):
    task: str
    sources: list[dict]
    draft: str
    approved: bool
    answer: str

def draft_node(state: ApprovalState) -> dict:
    sources = retrieve(collection, state["task"])
    pieces: list[str] = []
    for source in sources:
        pieces.append(source["text"])
    context = "\n\n".join(pieces)
```

```
    prompt = f"Context:\n{context}\n\nQuestion: {state['task']}"
    return {"sources": sources, "draft": model.invoke(prompt).content}

def source_files(state: ApprovalState) -> list[str]:
    files: list[str] = []
    for source in state["sources"]:
        files.append(source["source_file"])
    return files

def approval_node(state: ApprovalState) -> dict:
    decision = interrupt({
        "draft": state["draft"],
        "sources": source_files(state),
        "prompt": "Approve this answer before it's sent? (approved/rejected)",
    })
    return {"approved": decision == "approved"}

def finish_node(state: ApprovalState) -> dict:
    if state["approved"]:
        return {"answer": state["draft"]}
    return {"answer": "Not approved -- nothing was sent."}

def build_graph():
    builder = StateGraph(ApprovalState)
    builder.add_node("draft_node", draft_node)
    builder.add_node("approval_node", approval_node)
    builder.add_node("finish_node", finish_node)
    builder.add_edge(START, "draft_node")
    builder.add_edge("draft_node", "approval_node")
    builder.add_edge("approval_node", "finish_node")
    builder.add_edge("finish_node", END)
    return builder.compile(checkpointer=MemorySaver())

def run_with_approval(graph, task: str, decision, thread_id: str) -> dict:
    config = {"configurable": {"thread_id": thread_id}}
    graph.invoke({"task": task}, config)
    return graph.invoke(Command(resume=decision), config)

# Run the same task twice: once approving, once rejecting.
graph = build_graph()
task = "How hot should the water be when brewing?"
approved = run_with_approval(graph, task, "approved", "thread-yes")
print("approved run:", approved["answer"])
rejected = run_with_approval(graph, task, "rejected", "thread-no")
print("rejected run:", rejected["answer"])
```

Expected output:

```
approved run: Water for brewing coffee should be between 195 and 205
degrees Fahrenheit.
rejected run: Not approved -- nothing was sent.
```